

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.М41-мм

Разработка Linux-драйвера для RISC-V N-Trace подсистемы

Патов Артемий Сергеевич

Отчёт по производственной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к.ф.-м.н., В. И. Гориховский

Консультант:
эксперт по разработке ПО, ООО «КНС Групп», С. И. Матюкевич

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор систем аппаратной трассировки и механизмов их конфигурации	5
2.1. Спецификации аппаратной трассировки RISC-V	5
2.2. Возможности подсистемы профилирования perf	9
2.3. Обзор существующих решений: Intel PT и ARM Coresight ETM	10
2.4. Обзор open-source драйверов rvtrace	12
2.5. Вывод из обзора и постановка фаз реализации	13
3. Разработка драйвера	15
3.1. Поддержка регистров контекста	15
3.2. Архитектура настройки фильтров и компараторов	16
3.3. Интеграция адресных фильтров с подсистемой perf	16
3.4. Пример корректного исполнения запроса трассировки с фильтрацией	17
Заключение	20
Список литературы	22
А. Приложение: лог записи регистров драйвера rvtrace	24

Введение

Стремительное развитие современных вычислительных систем и постоянное усложнение программного обеспечения требуют создания эффективных инструментов для анализа производительности и отладки. Одним из наиболее мощных методов такого анализа является аппаратная трассировка, позволяющая детально отслеживать процесс исполнения кода с минимальными накладными расходами для самой системы. Для архитектуры RISC-V [13] в настоящее время активно развиваются стандарты трассировки, в частности спецификации RISC-V Trace Control Interface [15] и RISC-V N-Trace [14], предоставляющие богатые возможности для сбора данных, их сжатия и фильтрации.

Несмотря на наличие стандартов, интеграция аппаратных механизмов трассировки RISC-V с операционными системами, такими как Linux, все еще находится в стадии активной разработки и адаптации. Стандартная подсистема профилирования Linux (perf [9]) предоставляет базовые интерфейсы управления, однако гибкость и сложность архитектуры N-Trace требуют создания специализированных драйверов и расширенных механизмов конфигурации, что делает тему данной дипломной работы крайне актуальной.

Open source драйвер RISC-V Trace-системы [10] находится в зачаточном состоянии: в нем реализован базовый скелет интегрированного набора драйверов, платформенный драйвер, регистрирующий в ядре Linux аппаратные составляющие трейс-системы, описанные в Device Tree [2] с определенным набором свойств, поддержана минимальная функциональность для работы через perf-tool — создание perf-event'ов для включения и отключения всех необходимых компонентов в системе, формирующих граф потока трейс-данных. В рамках текущей учебной практики в качестве первого этапа планируется доработка существующего open source драйвера в части фильтрации по адресам и режимам привилегий.

1. Постановка задачи

Целью дипломной работы является разработка драйвера для системы трассировки N-Trace в архитектуре RISC-V, обеспечивающего интеграцию с подсистемой профилирования perf операционной системы Linux и предоставляющего механизмы конфигурации аппаратных компонентов трассировки.

На текущий семестр были поставлены следующие задачи:

1. Исследование спецификаций RISC-V-Trace-Control-Interface [15], RISC-V N-Trace [14] и анализ существующих систем аппаратной трассировки (Intel PT [3], ARM CoreSight ETM [1]);
2. Портирование и адаптация открытой реализации минимального драйвера rvtrace (патч `riscv_trace_support_v4` [10]) для целевой платформы, включая устранение выявленных программных недочетов;
3. Поддержка фильтрации по потоку инструкций и идентификаторам процессов:
 - Портирование патча, поддерживающего использование регистров контекста (`scontext`, `hcontext`) [12] в ядре Linux, в т.ч. для точной идентификации процессов операционной системы;
 - Разработка независимой настройки аппаратных фильтров и компараторов;
 - Обеспечение интеграции адресных фильтров с интерфейсом подсистемы perf [7].

2. Обзор систем аппаратной трассировки и механизмов их конфигурации

Перед разработкой драйвера N-Trace для RISC-V необходимо провести детальный анализ существующих спецификаций и программных реализаций, в том числе, для других архитектур.

2.1. Спецификации аппаратной трассировки RISC-V

Развитие аппаратной трассировки для архитектуры RISC-V опирается на стандарты RISC-V Trace Control Interface [15] и RISC-V N-Trace (Nexus-based Trace) [14]. История разработки спецификаций показывает, что многие архитектурные решения были заимствованы из экосистемы ARM CoreSight¹, что обуславливает схожесть их инфраструктуры. Например, наличие ATB Bridge в спецификации RISC-V напрямую указывает на желание обеспечить совместимость с уже существующей экосистемой ARM (см. рис. 1).

Спецификации детально описывают граф компонентов в железе (см. рис. 2), который включает в себя:

1. **Ingress Port** — интерфейс между ядром (hart) и системой трассировки, в который поступают значения с различных CSR регистров, например, `scontext`, `xtval`, `ecause` и др.
2. **Trace Encoder** — компонент, который принимает данные об исполнении от ядра через Ingress Port, сжимает её и форматирует в поток сообщений трассировки. На одном ядре не может быть больше одного Trace Encoder.
3. **Trace Funnel** (агрегатор) — компонент, объединяющий потоки трассировки от нескольких источников (энкодеров или других агрегаторов) в один выходной поток.

¹В ревью спецификации очень часты упоминания ARM Coresight: <https://lists.riscv.org/g/tech-nexus/message/328>

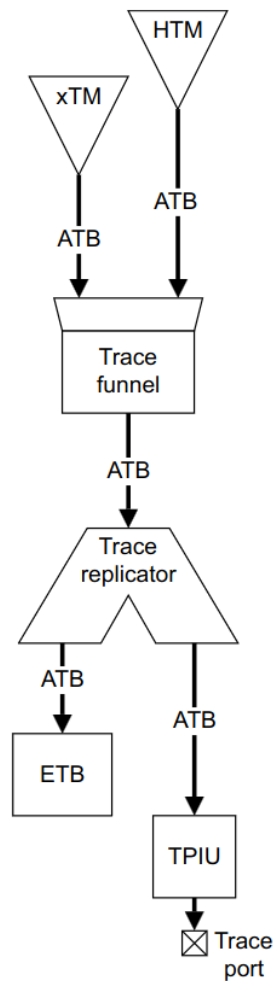


Рис. 1: Интеграция с инфраструктурой Coresight посредством ATB Bridge

4. **Trace Sink** (приемник) — конечный компонент, который принимает поток сообщений и сохраняет его в памяти (SRAM или системная память).
5. **ATB Bridge** — мост, позволяющий передавать данные трассировки RISC-V в инфраструктуру ARM CoreSight.

Возможности аппаратного энкодера охватывают широкий спектр настроек, которые напрямую отражены в регистровой модели, описанной в спецификации Trace Control Interface:

- Развитая система фильтрации: количество фильтров и компараторов, поддерживаемых энкодером определяется реализацией системы. Каждый фильтр может быть настроен на соответствие

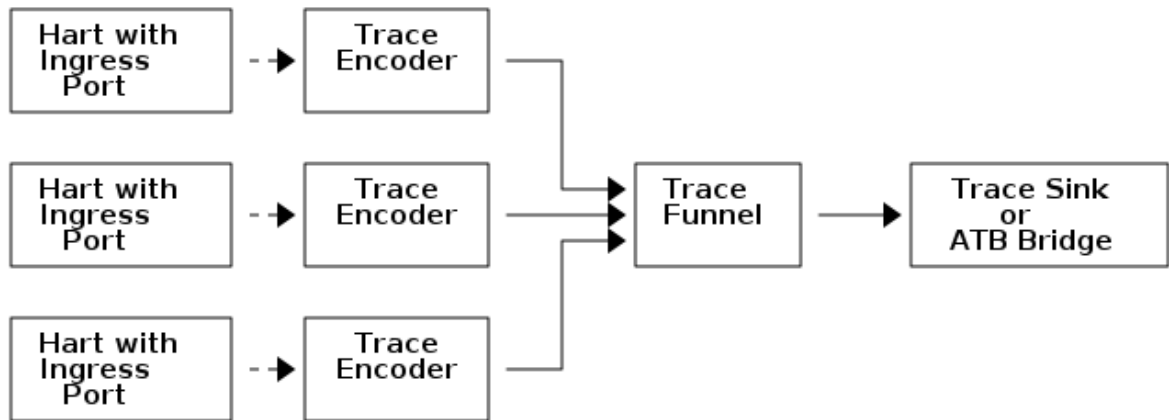


Рис. 2: Компоненты, через которые проходит поток trace-данных

по привилегиям (`trTeFilterMatchPrivilege`), кодам исключений (`trTeFilterMatchEcaue`), прерываниям (`trTeFilterMatchInterrupt`), типам данных (`trTeFilterMatchDtype`) и размерам данных (`trTeFilterMatchDsiz`). Компараторы позволяют сравнивать адреса инструкций (`iaddr`), контекст (`context`), значение исключения (`tval`) или адрес данных (`daddr`). Каждый фильтр может использовать до трех компараторов, редуцируя их результаты при помощи логического И (см. рис. 3).

- Режимы компрессии сообщений: Протоколы кодирования N-Trace поддерживают алгоритмы Branch Trace Messaging (BTM) и History Trace Messaging (HTM), обеспечивающие высокую степень сжатия потока исполнения. Выбор режима управляется полем `trTeInstMode`.
- Контекстные регистры: Через Ingress Port передаются значения специфичных CSR-регистров, таких как `scontext`, в котором хранят PID текущего процесса, и `hcontext`, в который записывают ID гостевой ОС при использовании гипервизора. Эти значения кодируются в сообщениях Ownership, что позволяет декодеру идентифицировать контекст выполнения.

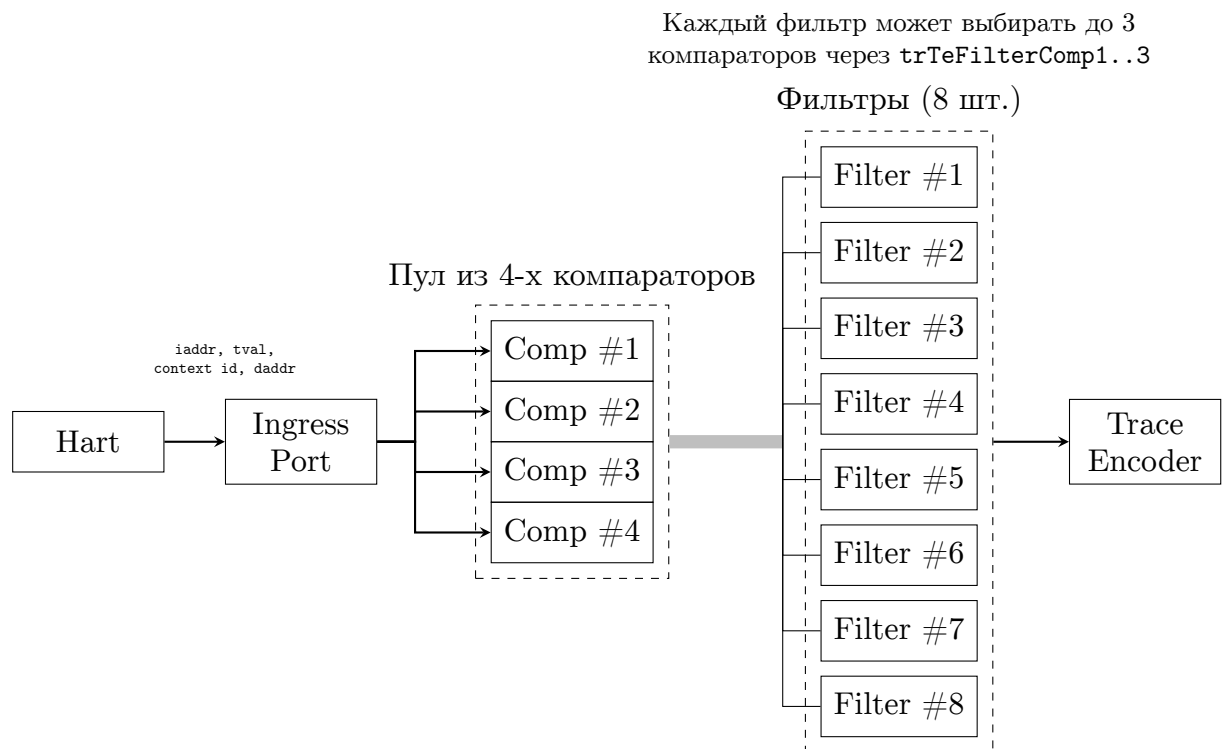


Рис. 3: Архитектура фильтрации N-Trace: сигналы Ingress Port распределяются по всем компараторам, результаты которых передаются по общей шине. Каждый из 8 фильтров может настраиваемым образом подключаться к любому из 4 компараторов (или нескольким), используя регистры выбора. Результаты используемых фильтров объединяются логическим ИЛИ.

2.2. Возможности подсистемы профилирования perf

Стандартным инструментом для управления трассировкой в ядре Linux является подсистема perf. Основное взаимодействие с драйверами (PMU) происходит через структуру `perf_event_attr`², которая содержит конфигурационные поля `config`, `config1`, `config2` и `config3`, в совокупности дающие 256 бит для передачи настроек аппаратному обеспечению [5]. В драйвере Intel PT эти поля используются для настройки различных аспектов трассировки.

Утилита perf предоставляет высокоуровневый интерфейс для управления трассировкой:

- Указание пропускаемых уровней привилегий через суффиксы `<event-name>//[u,k,hv]` [8], которые транслируются в поля `exclude_user`, `exclude_kernel` и `exclude_hypervisor` структуры `perf_event_attr`. Система трассировки RISC-V предоставляет возможность фильтрации еще при поступлении данных в Ingress Port.
- Опция `--pid` гарантирует включение ивента исключительно для указанных процессов, отключая его при планировщике для всех остальных. Это достигается путем фильтрации на уровне PMU, что позволяет избежать сбора данных от нецелевых процессов.
- Флаг `--filter` инициирует вызов `ioctl` с командой `PERF_EVENT_IOC_SET_FILTERS`, что используется для конфигурации аппаратных адресных фильтров. В драйверах CoreSight и Intel PT эта опция используется для настройки адресных диапазонов трассировки.
- Опция `-S` (`--snapshot-mode`) доставляется PMU-драйверу для работы в режиме скользящего окна ограниченного размера.

²`perf_event_attr`: https://github.com/torvalds/linux/blob/master/include/linux/perf_event.h

2.3. Обзор существующих решений: Intel PT и ARM Coresight ETM

В контексте разработки драйвера необходимо провести анализ существующих реализаций в ядре Linux — intel-pt [6] и coresight-etm [5], чтобы выявить их сходства и отличия от целевой архитектуры rvtrace [10].

2.3.1. Аппаратная инфраструктура

Intel Processor Trace (PT) представляет собой широко распространённое решение для архитектуры x86, обеспечивающее высокую пропускную способность трассировки при умеренной аппаратной сложности. Ключевой особенностью Intel PT является его относительная простота с точки зрения организации графа данных: трассировка генерируется непосредственно процессором и направляется в буфер памяти или экспортируется через интерфейс. Аппаратная фильтрация ограничена адресными диапазонами (фильтрация по CR3 и IP), а вся конфигурация полностью укладывается в стандартный API подсистемы perf через поля структуры perf_event_attr [7]. Это делает Intel PT удобным для интеграции, но существенно ограничивает его гибкость в сложных многоядерных системах.

В отличие от Intel PT, **ARM CoreSight ETM** предоставляет полноценную инфраструктуру трассировки, построенную по принципу графа компонентов. Поток данных в CoreSight выглядит следующим образом:

$$\text{CPU Core} \rightarrow \text{ETM} \xrightarrow{\text{ATB}} \text{Trace Funnel} \xrightarrow{\text{ATB}} \text{ETB} / \text{TPIU}$$

где:

- **ETM (Embedded Trace Macrocell)** — непосредственно источник трассировки, встроенный в ядро процессора;
- **ATB (AMBA Trace Bus)** — стандартизированная шина для маршрутизации трассировочных потоков между компонентами;

- **Trace Funnel** — агрегатор, объединяющий потоки от нескольких источников (например, от разных ядер) в один;
- **ETB (Embedded Trace Buffer)** или **TPIU (Trace Port Interface Unit)** — конечные приёмники (sinks), сохраняющие трассу в буфере памяти или экспортирующие её через физический порт.

Помимо сложной маршрутизации, CoreSight ETM предоставляет развитую систему фильтрации. Каждый ETM поддерживает несколько компараторов (до 8 в зависимости от реализации), которые могут настраиваться на:

- адресную фильтрацию (include/exclude по диапазонам адресов инструкций);
- фильтрацию по адресам данных;
- контекстную фильтрацию (по идентификаторам процессов, VMID);
- фильтрацию по типам доступа (чтение/запись).

При этом фильтры могут быть скомбинированы в сложные логические выражения, что обеспечивает высокую гибкость при отборе трассируемых участков кода. Однако такая гибкость требует выхода за рамки стандартного perf API: в CoreSight расширенные настройки производятся через интерфейсы `sysfs` и `configs`, а не через поля `perf_event_attr`.

Проведённый анализ показывает, что архитектура RISC-V N-Trace имеет прямые параллели с ARM CoreSight:

- **Граф компонентов:** RISC-V N-Trace также включает Trace Encoder (аналог ETM), Trace Funnel (агрегатор) и Trace Sink (приёмник), соединённые через стандартизированные интерфейсы (в случае RISC-V — через внутренние шины или ATB Bridge для совместимости с CoreSight).

- **Система фильтрации:** как и CoreSight, RISC-V N-Trace предоставляет пул компараторов, которые могут быть назначены фильтрам для адресной, контекстной и др. фильтрации, но в отличие от Coresight компараторы и фильтры — отдельные сущности.

Таким образом, из двух рассмотренных решений архитектура RISC-V N-Trace оказывается значительно ближе к ARM CoreSight, чем к Intel PT. Это обуславливает выбор CoreSight в качестве основного референса при разработке драйвера `rvtrace`.

2.3.2. Linux-драйвер ARM Coresight ETM

Код драйвера `coresight-etm4x-core.c` [4] демонстрирует использование механизмов `configs` и загрузки модулей ядра для конфигурации. В частности, в `coresight-etm4x-core.c` есть функции для работы с `sysfs` и `configs`, а также модульные параметры для настройки поведения при загрузке. Использование загрузки модуля с конфигурацией признано потенциально небезопасным, поэтому предпочтительнее фиксировать параметры в `sysfs`. Такой подход увеличивает безопасность за счет возможности задавать жесткие ограничения на объем сохраняемых данных. Однако использование `sysfs` несет существенный недостаток: применяемая конфигурация глобальна для всей системы. При необходимости одновременного запуска нескольких эксклюзивных ивентов на одном процессоре управление ими ложится на плечи пользователя, поскольку полностью полагаться на внутренний код и диспетчеризацию `perf` в данном случае невозможно. Очевидно, что такие события профилирования должны работать исключительно в монопольном режиме (флаг `exclusive`).

2.4. Обзор open-source драйверов `rvtrace`

В рамках исследования изучены подходы к реализации драйвера `rvtrace`, в частности, патч-серия, представленная в upstream-ядре Linux [10]. Драйвер `rvtrace` состоит из нескольких модулей, каждый из которых отвечает за конкретную функциональную область:

- `rvtrace-platform.c` — platform-драйвер, обеспечивающий обнаружение и инициализацию различных компонентов системы трассировки (энкодеров, фанналов, синков) на основе данных из Device Tree;
- `rvtrace-core.c` — ядро драйвера, предоставляющее внутренние абстракции для работы с компонентами железа, управление путями трассировки (trace paths) и функции их запуска/остановки;
- `rvtrace-encoder.c` — драйвер управления энкодером, реализующий конфигурацию аппаратных регистров, фильтров и компараторов;
- `rvtrace-perf.c` — интеграция с подсистемой `perf`, регистрация PMU-события и обработка вызовов из пользовательского пространства;
- `rvtrace-ramsink.c` — управление RAM-синком, обеспечивающее запись трассировочных данных в выделенную область памяти.

Для поддержки работы с контекстными регистрами, такими как `scontext` и `hcontext`, определёнными в спецификации Sdtrig [12], в open source существует серия патчей [11]. Эти регистры предназначены для идентификации контекста выполнения при трассировке и позволяют декодеру связывать поток инструкций с конкретными процессами операционной системы. В спецификации N-Trace [14] предусмотрена их интеграция с механизмом генерации сообщений Ownership.

2.5. Вывод из обзора и постановка фаз реализации

Анализ показал, что стандартный API `perf` предоставляет базовые возможности конфигурирования, однако `rvtrace` обладает кратно большим числом степеней свободы. Для некоторых групп заказчиков, планирующих выполнять глубокую трассировку, критически важна возможность полной настройки всех аппаратных компонентов (encoder'ов) отдельно для каждого процессора в многопроцессорных системах.

Исходя из этих предпосылок, поддержка конфигурации трейс-системы в разрабатываемом драйвере разбивается на две последовательные фазы:

- Минимальный прототип. Включает в себя поддержку адресных фильтров, режимов привилегий (u, k, hv) и режима `–snapshot-mode`. Весь указанный функционал полностью укладывается в существующие опции утилиты и API perf.
- Полная конфигурация. Предоставляет исчерпывающую настройку энкодеров (encoder), которая может индивидуально отличаться для каждого процессора. Эта функциональность реализуется в обход стандартного API perf при помощи механизмов ядра Linux, таких как sysfs или загрузка отдельного модуля (что является менее предпочтительным из-за вопросов безопасности).

3. Разработка драйвера

Основой для практической части курсовой работы послужила открытая реализация драйвера `rvtrase` [10]. В процессе адаптации и расширения функциональности драйвера для целевой платформы был выполнен ряд задач, направленных на повышение стабильности, обеспечение корректной работы с контекстом исполнения и добавление новых возможностей управления трассировкой.

Первым этапом практической реализации стала доработка исходного кода базовой версии драйвера. В ходе детального анализа логики работы были выявлены и устранены программные ошибки. Одна из таких ошибок заключается в следующем. При запуске `perf-event` создается список компонентов, который необходимо активировать, и возвращается указатель на соответствующую структуру. В случае ошибки возвращается код, имеющий тип указателя, сформированный при помощи макроса `ERR_PTR`. Из-за неправильной обработки ошибок невозможность доступа к компонентам в конечном итоге приводила к `kernel panic`. Исправление этого недочета повысило отказоустойчивость драйвера.

3.1. Поддержка регистров контекста

Для обеспечения точной привязки потока трассировки к процессам операционной системы был портирован и существенно доработан ранний программный патч, добавляющий поддержку регистров контекста [11]. Интеграция управляющих регистров `scontext` и `hcontext` позволяет аппаратному обеспечению прозрачно получать идентификаторы текущего процесса и гостевой операционной системы. Эти данные необходимы для корректного декодирования и последующего анализа собранной информации, исключая смешивание данных от разных потоков исполнения.

3.2. Архитектура настройки фильтров и компараторов

Ключевой задачей разработки стало расширение модуля `rvtrace-encoder` для поддержки гибкой и независимой конфигурации аппаратных фильтров. Архитектура драйвера была переработана таким образом, чтобы фильтры и компараторы представляли собой независимые программные сущности с четко определенным жизненным циклом. Подобное разделение обеспечивает необходимый уровень абстракции и решает сразу две стратегические задачи:

- Универсальность базовой настройки: Обеспечивается возможность корректной установки минимального набора параметров фильтрации стандартными средствами подсистемы `perf`.
- Масштабируемость решения: Закладывается архитектурный фундамент для реализации второй фазы конфигурации. Выделение независимых сущностей позволит в будущем осуществлять полный доступ ко всем тонким настройкам аппаратного источника с использованием специализированных механизмов ядра (например, через файловую систему `sysfs` или загружаемые модули), не нарушая при этом общую целостность драйвера.

3.3. Интеграция адресных фильтров с подсистемой `perf`

Заключительным этапом текущей фазы разработки стала модификация `rvtrace-perf`. В код драйвера была добавлена полноценная поддержка адресных фильтров. Это нововведение позволяет пользователю задавать конкретные диапазоны памяти для профилирования непосредственно через стандартные команды интерфейса без использования сторонних утилит.

Для реализации этой функции было организовано прямое взаимодействие между интерфейсным модулем `rvtrace-perf` и управляющим

Листинг 1: Команда запуска трассировки с фильтрацией

```
perf record -e rvtrace/event=0x1/uk \
  --filter 'filter 0xffff / 0xffffffff \
    filter 0x1234 / 0x3210 \
    start 0xC \
    stop 0xffffffffffff0000 \
    filter 0xffff / 0xffffffff \
    filter 0x1234 / 0x3210 \
    start 0xC \
    stop 0xffffffffffff0000' \
perf bench sched pipe -l 100
```

модулем rvtrace-encoder.

3.4. Пример корректного исполнения запроса трассировки с фильтрацией

Для верификации корректности трансляции высокоуровневых опций `perf` в аппаратные регистры N-Trace Encoder был проведён эксперимент с использованием команды [8], которую можно увидеть в листинге 1.

Параметр `event=0x1` включает базовую инструкционную трассировку, а суффикс `uk` устанавливает фильтрацию по уровням привилегий: трассировка ведётся для пользовательского режима (`u`) и режима ядра (`k`), исключая гипервизор (`hv`). Опция `--filter` задаёт адресные фильтры, которые транслируются в аппаратные компараторы и регистры управления фильтрами энкодера.

Для отладки в драйвер `rvtrace` был добавлен вывод всех операций записи в управляющие регистры. Полученный лог (см. приложение А) показывает последовательность конфигурирования регистров энкодера.

3.4.1. Интерпретация записей

Спецификация [15] определяет, что регистры фильтров расположены по смещениям 0x400..0x5FF, а компараторов — 0x600..0x6FF относительно базового адреса энкодера. Анализ лога показывает следующее:

- Записи по смещениям 0x400, 0x420, 0x440, 0x460, 0x480, 0x4A0, 0x4C0, 0x4E0 соответствуют регистрам управления фильтрами `trTeFilteriControl`. Значения 0x13, 0x33, 0x53, 0x73, 0x93, 0xB3, 0xD3, 0xF3 кодируют включение фильтра и настройку сравнений (например, бит `trTeFilterMatchPrivilege`, `trTeFilterMatchComp1` и т.д.).
- По смещению 0x4E4 записано значение 0x63 (последняя строка лога), что соответствует настройке фильтра #7.
- Регистры компараторов (смещения 0x600..0x6FF) настраиваются парами для первичного и вторичного сравнения. Например, для первого компаратора:
 - 0x610: `trTeCompPMatchLow` = 0xFFFF.
 - 0x614: `trTeCompPMatchHigh` = 0x0.
 - 0x618: `trTeCompSMatchLow` = 0xFFFE.
 - 0x61C: `trTeCompSMatchHigh` = 0x1.
 - 0x600: `trTeCompControl` = 0x1250 (кодирует выбор входов и функций сравнения).
- Аналогично настраиваются компараторы #1 и #2 (смещения 0x630..0x67C и 0x690..0x6FC), что соответствует двум парам фильтров из команды.
- Значения `start` и `stop` (0xC и 0xFFFFFFFFFFFFFFFF0000) записываются в регистры диапазонов (`trTeFilteriMatchEcaUseLow/High` или `trTeCompjPMatchLow/High` в зависимости от реализации), что видно по записям в 0x650..0x65C и 0x670..0x67C.

3.4.2. Выводы

Эксперимент демонстрирует, что механизм `perf` позволяет передавать конфигурации фильтров через опцию `--filter`. Драйвер `rvtrace` успешно отображает эти высокоуровневые параметры на регистры компараторов и фильтров.

Заключение

В ходе выполнения учебной практики были успешно выполнены следующие задачи:

- Были изучены релевантные для настоящей работы разделы спецификаций Intel PT и ARM Coresight ETM, а также код их драйверов в ядре Linux. Было решено при реализации драйвера rvtrace ориентироваться на ARM Coresight ETM из-за схожести аппаратных инфраструктур;
- Открытый патч с минимальной реализацией драйвера rvtrace был перенесен в проприетарный репозиторий, некоторые недостатки были исправлены;
- Поддержка фильтрации по потоку инструкций и идентификаторам процессов:
 - Портирован патч, поддерживающий использование регистров контекста. Регистры scontext/hcontext в настоящий момент успешно используются для идентификации процесса, к которому относятся трассировочные данные.
 - Поддержана независимая настройка фильтров и компараторов в драйвере rvtrace-encoder.
 - Обеспечена интеграция адресных фильтров с интерфейсом подсистемы perf.

Представленный в работе программный код был разработан в соответствии с техническим заданием и проверен консультантом. Положительный отзыв консультанта о результатах работы имеется. В силу действия соглашения о неразглашении текст кода не может быть включен в открытую часть отчета.

В силу того, что поддержку фильтрации было решено проводить в две фазы, а в ходе настоящей учебной практики запланировано выполнить только первую, вторая, т. е. предоставление возможности пользо-

вателю полностью конфигурировать фильтры и компараторы, остается одной из задач на следующий семестр.

Также предстоит поддержать в драйвере snapshot-режим, который контролируется на уровне аппаратуры.

Одна из объемных будущих задач, следующих за поддержкой в драйвере базовой функциональности, описанной в спецификации — это экспорт трассировочных данных через PCIe.

Список литературы

- [1] Arm Limited. — CoreSight ETM-M4 Technical Reference Manual. Revision r0p1, 2010. — ARM DDI 0440C.
- [2] Devicetree.org. — Devicetree Specification, Release v0.4, 2023. — URL: <https://github.com/devicetree-org/devicetree-specification/releases/tag/v0.4> (дата обращения: 5 июля 2026 г.).
- [3] Intel Corporation. — Intel 64 and IA-32 Architectures Software Developer’s Manual. Volume 3C: System Programming Guide, Part 3. Chapter 32: Intel Processor Trace, 2026. — URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> (дата обращения: 5 июля 2026 г.).
- [4] Linux Kernel Organization. CoreSight ETMv4.x hardware tracing driver core. — URL: <https://github.com/torvalds/linux/blob/master/drivers/hwtracing/coresight/coresight-etm4x-core.c> (дата обращения: 5 июля 2026 г.).
- [5] Linux Kernel Organization. CoreSight Hardware Tracing Subsystem Drivers. — URL: <https://github.com/torvalds/linux/tree/master/drivers/hwtracing/coresight> (дата обращения: 5 июля 2026 г.).
- [6] Linux Kernel Organization. Intel Processor Trace (PT) Perf Event Implementation. — URL: <https://github.com/torvalds/linux/blob/master/arch/x86/events/intel/pt.c> (дата обращения: 5 июля 2026 г.).
- [7] Linux Kernel Organization. Performance Counters for Linux (Design document). — URL: <https://github.com/torvalds/linux/blob/master/tools/perf/design.txt> (дата обращения: 5 июля 2026 г.).

- [8] Linux manual pages. perf-record(1) — Linux profile tool man-page. — URL: <https://man7.org/linux/man-pages/man1/perf-record.1.html> (дата обращения: 5 июля 2026 г.).
- [9] Linux manual pages. perf(1) — Linux profile tool man-page. — URL: <https://man7.org/linux/man-pages/man1/perf.1.html> (дата обращения: 5 июля 2026 г.).
- [10] Patel Anup. Minimal rvtrace driver implementation (riscv_trace_support_v4 branch). — URL: https://github.com/avpatel/linux/tree/riscv_trace_support_v4/drivers/hwtracing/rvtrace (дата обращения: 5 июля 2026 г.).
- [11] Patel Anup. RISC-V Trace Infrastructure and N-Trace Support (Patch v1). — URL: <https://lore.kernel.org/linux-riscv/20240329-dev-maxh-lin-452-6-9-v1-0-1534f93b94a7@sifive.com/> (дата обращения: 5 июля 2026 г.).
- [12] RISC-V International. — RISC-V Debug Specification. Version 1.0, 2024. — URL: <https://github.com/riscv/riscv-debug-spec> (дата обращения: 5 июля 2026 г.).
- [13] RISC-V International. — The RISC-V Instruction Set Manual. Volume II: Privileged Architecture. Version 20240411, 2024.
- [14] RISC-V International. — RISC-V N-Trace (Nexus-based Trace) Specification. Version 1.0, 2024. — URL: <https://github.com/riscv/riscv-n-trace> (дата обращения: 5 июля 2026 г.).
- [15] RISC-V International. — RISC-V Trace Control Interface Specification. Version 1.0, 2024. — URL: <https://github.com/riscv/riscv-trace-control-interface> (дата обращения: 5 июля 2026 г.).

А. Приложение: лог записи регистров драйвера rvtrace

Листинг 2: Лог записи регистров драйвера rvtrace

```
[ 2394.963751] RVTRACE Write. val: f3, offset: 4e0
[ 2395.121687] RVTRACE Write. val: ffff, offset: 610
[ 2395.122072] RVTRACE Write. val: 0, offset: 614
[ 2395.122303] RVTRACE Write. val: fffe, offset: 618
[ 2395.122523] RVTRACE Write. val: 1, offset: 61c
[ 2395.122752] RVTRACE Write. val: 1250, offset: 600
[ 2395.122949] RVTRACE Write. val: 1234, offset: 630
[ 2395.123151] RVTRACE Write. val: 0, offset: 634
[ 2395.123369] RVTRACE Write. val: 4444, offset: 638
[ 2395.123582] RVTRACE Write. val: 0, offset: 63c
[ 2395.123808] RVTRACE Write. val: 1250, offset: 620
[ 2395.123975] RVTRACE Write. val: c, offset: 650
[ 2395.124191] RVTRACE Write. val: 0, offset: 654
[ 2395.124380] RVTRACE Write. val: 0, offset: 658
[ 2395.124581] RVTRACE Write. val: 0, offset: 65c
[ 2395.124786] RVTRACE Write. val: 50, offset: 640
[ 2395.124996] RVTRACE Write. val: ffff0000, offset: 670
[ 2395.125221] RVTRACE Write. val: ffffffff, offset: 674
[ 2395.125431] RVTRACE Write. val: 0, offset: 678
[ 2395.125606] RVTRACE Write. val: 0, offset: 67c
[ 2395.125749] RVTRACE Write. val: 20, offset: 660
[ 2395.125911] RVTRACE Write. val: ffff, offset: 690
[ 2395.126054] RVTRACE Write. val: 0, offset: 694
[ 2395.126202] RVTRACE Write. val: fffe, offset: 698
[ 2395.126344] RVTRACE Write. val: 1, offset: 69c
[ 2395.126491] RVTRACE Write. val: 1250, offset: 680
[ 2395.126632] RVTRACE Write. val: 1234, offset: 6b0
[ 2395.126775] RVTRACE Write. val: 0, offset: 6b4
```


[2395.126924]	RVTRACE Write. val: 4444, offset: 6b8
[2395.127062]	RVTRACE Write. val: 0, offset: 6bc
[2395.127208]	RVTRACE Write. val: 1250, offset: 6a0
[2395.127348]	RVTRACE Write. val: c, offset: 6d0
[2395.127493]	RVTRACE Write. val: 0, offset: 6d4
[2395.127633]	RVTRACE Write. val: 0, offset: 6d8
[2395.127774]	RVTRACE Write. val: 0, offset: 6dc
[2395.127920]	RVTRACE Write. val: 50, offset: 6c0
[2395.128060]	RVTRACE Write. val: ffff0000, offset: 6f0
[2395.128192]	RVTRACE Write. val: ffffffff, offset: 6f4
[2395.128345]	RVTRACE Write. val: 0, offset: 6f8
[2395.128502]	RVTRACE Write. val: 0, offset: 6fc
[2395.128641]	RVTRACE Write. val: 20, offset: 6e0
[2395.128782]	RVTRACE Write. val: 63, offset: 404
[2395.128969]	RVTRACE Write. val: 13, offset: 400
[2395.129183]	RVTRACE Write. val: 63, offset: 424
[2395.129398]	RVTRACE Write. val: 33, offset: 420
[2395.129609]	RVTRACE Write. val: 63, offset: 444
[2395.129832]	RVTRACE Write. val: 53, offset: 440
[2395.130042]	RVTRACE Write. val: 63, offset: 464
[2395.130263]	RVTRACE Write. val: 73, offset: 460
[2395.130477]	RVTRACE Write. val: 63, offset: 484
[2395.130695]	RVTRACE Write. val: 93, offset: 480
[2395.130914]	RVTRACE Write. val: 63, offset: 4a4
[2395.131125]	RVTRACE Write. val: b3, offset: 4a0
[2395.131340]	RVTRACE Write. val: 63, offset: 4c4
[2395.131555]	RVTRACE Write. val: d3, offset: 4c0
[2395.131769]	RVTRACE Write. val: 63, offset: 4e4